



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

Apr. 24th



Outline

- ◆ Increment and Decrement Operators
- ◆ While loop
- ◆ For loop

Increment and Decrement Operators

- ◆ C provides two special unary operators for incrementing and decrementing variables
 - ❖ Increment operator: ++, add 1
 - ❖ Decrement operator: --, subtract 1
- ◆ ++n (prefix) and n++ (postfix) are different
 - ❖ The expression ++n increments n before its value is used
 - ❖ The expression n++ increments n after its value has been used
- ◆ The increment and decrement operators can only be applied to variables
 - ❖ An expression like (i+j)++ is illegal

Case 1

```
#include <stdio.h>

int main()
{
    int i=1;
    int a;

    a=++i;

    printf("a=%d, i=%d\n", a, i);

    return 0;
}
```

```
#include <stdio.h>

int main()
{
    int i=1;
    int a;

    a=i++;

    printf("a=%d, i=%d\n", a, i);

    return 0;
}
```

Case 1 (cont.)

```
#include <stdio.h>

int main()
{
    int i=1;
    int a;

    a=++i;

    printf("a=%d, i=%d\n", a, i);

    return 0;
}
```

a=2, i=2

```
#include <stdio.h>

int main()
{
    int i=1;
    int a;

    a=i++;

    printf("a=%d, i=%d\n", a, i);

    return 0;
}
```

a=1, i=2

Case 2

```
#include <stdio.h>

int main()
{
    int i=1;

    printf("i=%d\\n", i++);

    return 0;
}
```

```
#include <stdio.h>

int main()
{
    int i=1;

    printf("i=%d\\n", ++i);

    return 0;
}
```

Case 2 (cont.)

```
#include <stdio.h>

int main()
{
    int i=1;

    printf("i=%d\\n", i++);

    return 0;
}
```

i=1

```
#include <stdio.h>

int main()
{
    int i=1;

    printf("i=%d\\n", ++i);

    return 0;
}
```

i=2

Outline

- ◆ Increment and Decrement Operators
- ◆ While loop
- ◆ For loop

While

While

- Loop- While

```
while ( expression ) {  
    statement1;  
    statement2;  
    ...  
    statementn;  
}
```

If the expression is true,
statement1, statement2, ... statementn
will be executed.

(The details of expression
are available in the 4th class)

```
int i = 1, sum = 0;
```

```
...
```

```
while ( i <= 10 ) {  
    sum = sum + i;  
    i = i + 1;  
}
```

```
printf( "%d\n", sum );
```

What is the answer?

Example of While Statement

```
int i = 1, sum = 0;  
...  
while (i <= 10) {  
    sum = sum + i;  
    i = i + 1;  
}  
printf("%d\n", sum);
```

i	Sum
1	0+1=1
2	1+2=3
3	3+3=6
4	6+4=10
5	10+5=15
6	15+6=21
7	21+7=28
8	28+8=36
9	36+9=45
10	45+10=55

Exercise 1

- ◆ Please use while to print 11, 12, 13, 14, 15, 16, 17, 18, 19

```
value of a: 11  
value of a: 12  
value of a: 13  
value of a: 14  
value of a: 15  
value of a: 16  
value of a: 17  
value of a: 18  
value of a: 19
```

Exercise 1 (cont.)

```
#include <stdio.h>

int main () {

    /* local variable definition */
    int a = 11;

    /* while loop execution */
    while( a < 20 ) {
        printf("value of a: %d¥n", a);
        a++;
    }

    return 0;
}
```

Outline

- ◆ Increment and Decrement Operators
- ◆ While loop
- ◆ For loop

for

a method to define a loop as while.

```
for ( initialization_expression; loop_condition; increment_expression) {  
    statement1;  
    statement2;  
    ...  
    statementn;  
}
```



- initialization_expression is executed before execution of the loop starts.
- The execution of the loop continues until the loop_condition is false.
- The increment_expression is executed at the end of each loop iteration.

Comparison of While and For Loop

While loop

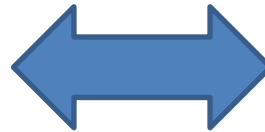
```
#include<stdio.h>

int main(void)
{
    int i = 1, sum = 0;

    while (i<=10) {
        sum=sum+i;
        i=i+1;
    }

    printf( "%d\n", sum );

    return 0;
}
```



for loop

```
#include<stdio.h>

int main(void)
{
    int sum = 0;

    for(int i=1; i<=10; i++)
    {
        sum = sum+i;
    }

    printf( "%d\n", sum );

    return 0;
}
```

Missing Expressions in the for loop

- ◆ Any or all of the expressions in a *for* statement can be missing, but **the two semicolons must remain**.

```
#include <stdio.h>

int main()
{
    int sum=0;
    for (int i = 1; i <=10 ; i++)
    {
        sum=sum+i;
    }

    printf("%d\n",sum);
}
```



```
#include <stdio.h>

int main()
{
    int sum=0;
    int i=1;
    for (; i <=10 ; )
    {
        sum += i++;
    }

    printf("%d\n",sum);
}
```

Exercise 2

- ◆ Please use for to print 11, 12, 13, 14, 15, 16, 17, 18, 19

```
value of a: 11  
value of a: 12  
value of a: 13  
value of a: 14  
value of a: 15  
value of a: 16  
value of a: 17  
value of a: 18  
value of a: 19
```

Exercise 2 (cont.)

```
#include <stdio.h>

int main () {

    int i;

    for(i=11;i<20;i++) {

        printf("i=%d¥n", i);

    }

    return 0;
}
```

Equivalence of for and while Loops

```
for (expr1; expr2; expr3)
```

```
statement  
statement
```

Is Equivalent to:

```
expr1;  
while (expr2) {  
    statement  
    expr3;  
}  
next statement
```

The *for* loop has the advantage of keeping both the control (*expr2*) and the indexing (*expr3*) all in one place at the top of the loop.

Break Statement

The break statement can be used in terminating loops (for, while)

```
While (expression){  
statement;  
.....
```

```
  If (expression) {  
    break;  
  }  
}
```



Example for The Break Statement

```
while(1)
{
    printf("enter a double > ");
    scanf("%lf", &x);
    if(x<0.0)
        break;
    printf("%lf\n",x );
}
```

```
/* break causes jump to next statement */
/* after the while loop.                */
```

Exercise 3

- ◆ Find the multiples of 3 from 1 to 100
 - ✦ Print out each value
 - ✦ Print out the summation of the multiples of 3

```
multiples of 3: 3
multiples of 3: 6
multiples of 3: 9
multiples of 3: 12
multiples of 3: 15
multiples of 3: 18
multiples of 3: 21
multiples of 3: 24
multiples of 3: 27
multiples of 3: 30
multiples of 3: 33
multiples of 3: 36
multiples of 3: 39
```

⋮

summation of these multiples is ?

Exercise 3 (cont.)

```
#include<stdio.h>

int main(void)
{
    int a = 1;
    int sum = 0;

    for(int a=1; a<=100; a++)
    {
        if(a%3==0)
        {
            printf("multiples of 3: %d\n", a);
            sum += a;
        }
    }

    printf("summation of these multiples is %d\n", sum);

    return 0;
}
```